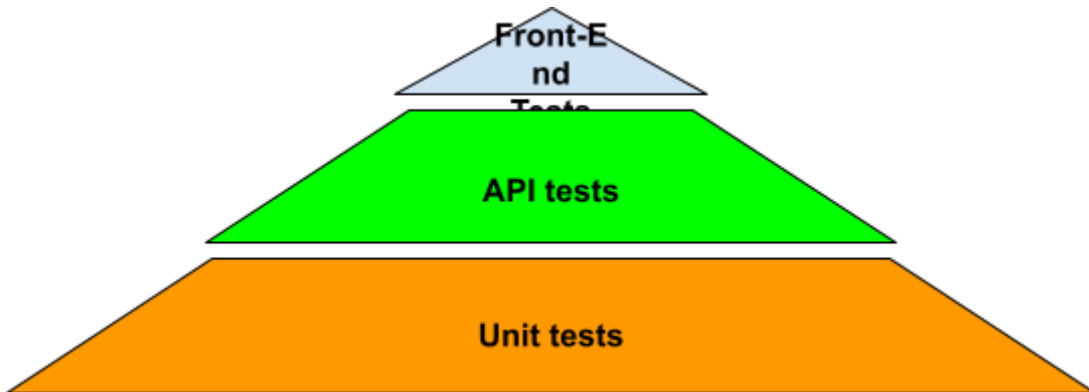


Análise da Automação de Testes em Arquiteturas Multicamadas

1. Contexto

A automação de testes em sistemas modernos ocorre em múltiplas camadas, tradicionalmente organizadas segundo a Pirâmide de Testes, conceito popularizado por Mike Cohn.

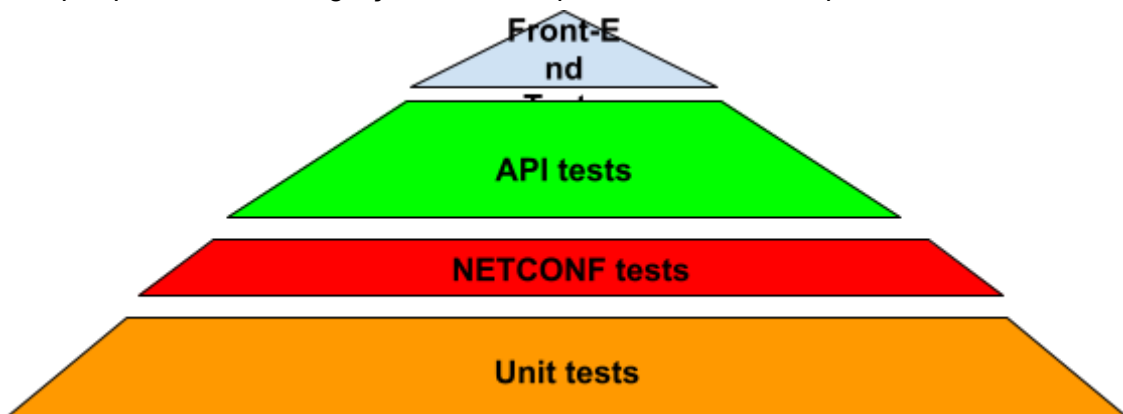


Essa abordagem recomenda:

- Maior volume de testes em níveis inferiores (unitários e de serviço)
- Menor quantidade de testes na camada de interface gráfica (E2E)

Esse é um conceito compatível com a definição de Agile Testing.

Na minha experiência, embora os testes unitários fossem majoritariamente responsabilidade do time de desenvolvimento, participei ativamente sugerindo cenários de teste, revisando cobertura e contribuindo em estratégias de validação antecipada. Seguindo mesmo espírito, porém com atuação e responsabilidade mais abrangentes, trabalhei fortemente na parte do meio onde normalmente começa a integração com a Gerência e o SW embarcado, e, no topo da pirâmide, obviamente, na validação E2E. Porém, minha pirâmide ainda era um pouco diferente dessa, porque antes da integração ainda era possível realizar um passo antes.



Especificamente, nos projetos em que trabalhei, uma parte do esforço da automação era direcionada a contribuir na parte em “vermelho” da pirâmide. Como ConfD (Framework da Tail-F) já fornecia uma interface que permitia fazer uma variedade de aplicações, inclusive uma interface NETCONF/YANG para automação, a atuação nessa parte da pirâmide foi utilizada como uma espécie de reforço aos testes

unitários do Software Embarcado. Ou seja, essa validação ocorria antes da integração com a Gerência ou quando não havia a necessidade de uma integração específica com a Gerência.

Entretanto, havia discussões sobre o desbalanceamento frequente dessa estrutura. Frameworks amplamente utilizados, como Selenium, Playwright e RestAssured, viabilizam tecnicamente a automação nas diferentes camadas, mas não resolvem problemas estruturais da estratégia de testes.

OBS.: API REST é um dos focos aqui porque o Back-End é um conjunto de subcamadas muito abrangente (banco de dados, mensageria, segurança, performance). Portanto, em se tratando de API, o objetivo é validar regras de negócio, integração e consistência de dados no contexto dos serviços para garantir que as regras de negócio funcionam corretamente.

2. Problema

Em ambientes reais de desenvolvimento, surgem desafios recorrentes como:

- Crescimento excessivo de testes E2E
- Alta instabilidade (flakiness) na camada de interface
- Tempo de execução longa em pipelines CI/CD
- Alto custo de manutenção
- Redundância entre testes de API e E2E
- Falta de critérios objetivos para distribuição dos testes entre camadas

Outros pontos a serem discutidos são com relação à cultura típica das empresas com relação à estruturação da pirâmide de testes:

- A definição da estratégia de testes costuma ser estática
- A priorização de execução raramente considera dados históricos
- A redistribuição da pirâmide ocorre de forma reativa, não sistemática

3. Possíveis questões a serem respondidas

- Como medir objetivamente o equilíbrio ideal da pirâmide de testes?
- Quais métricas indicam sobrecarga inadequada na camada de UI?
- Como reduzir flakiness sem comprometer cobertura funcional?
- Como evitar redundância entre testes de API e E2E?
- Como tornar a execução de testes mais eficiente em ambientes CI/CD?

4. “O que fazer?”

Em discussão. No meu entendimento, o tema é bastante abrangente e há vários caminhos que poderiam ser seguidos, como:

- Ter um ambiente experimental controlado para obter métricas relacionadas às questões levantadas anteriormente;
- Discutir estratégias para otimização da automação e escolher algumas delas para se aprofundar;

- Combinar os dois itens anteriores e obter no final algo até simulado demonstrando soluções para as questões levantadas anteriormente;
- ETC...